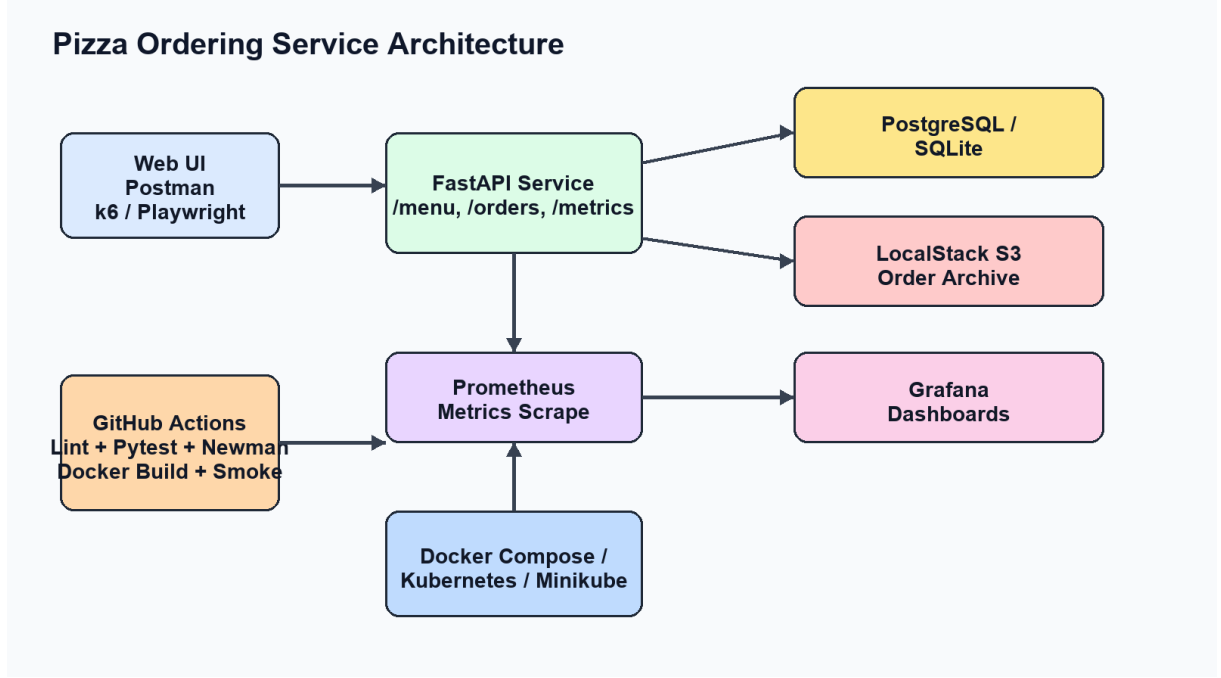


Pizza Ordering Service Final Raporu

Bu proje, küçük bir pizza sipariş domain'i üzerinden test mühendisliği, CI/CD, container, gözlemlenebilirlik ve dağıtım pratiklerini bir araya getiren bir backend uygulaması olarak geliştirilmiştir.



1. Giriş

Projenin temel hedefi karmaşık bir iş alanı modellemek değil, sınırlı kapsamlı bir servis etrafında endüstri standardına yakın bir teslim süreci oluşturmaktır.

Bu kapsamda FastAPI ile REST servisi geliştirilmiş, siparişlerin veritabanında tutulması, LocalStack S3 üzerinde arşivlenmesi ve test altyapısıyla doğrulanması sağlanmıştır.

2. Mimari Bile enler

Sistem FastAPI tabanlı bir uygulama, PostgreSQL veya SQLite veritabanı, LocalStack S3, Prometheus ve Grafana bile enlerinden oluşmaktadır.

GitHub Actions, lint, test, Docker build, Newman ve smoke test adımlarını çalıştırarak CI hattını oluşturmaktadır.

Yerel geliştirme için Docker Compose kullanılmı , dağıtım hedefi olarak ise Kubernetes manifestleri hazırlanmıştır.

3. Test Stratejisi

Unit testler fiyat hesaplama, kupon uygulama, durum geçişleri ve API davranışlarını doğrulamaktadır.

Integration testler PostgreSQL ve LocalStack üzerinden gerçek persistence ve arşivleme davranışlarını kontrol etmektedir.

Playwright için temel E2E omurgası hazırlanmış, kapsamlı ortamlarda suite'i k rmadan skip olacak şekilde düzenlenmiştir.

Postman ve Newman koleksiyonu ile endpoint sıralı doğrulama yapılmakta, coverage e i i CI tarafında en az yüzde 70 olarak zorunlu tutulmaktadır.

4. API Kapsamı ve Teknik Çıktılar

Serviste sağık kontrolü, menü listeleme, sipari oluşturma, sipari listeleme, sipari detay görüntüleme, sipari durumu güncelleme ve kupon uygulama uçları yer almaktadır.

Factory Boy ve Faker ile test verisi üretimi için `MenuItemFactory`, `OrderItemFactory` ve `OrderFactory` tanımlanmıştır.

Kubernetes tarafında Deployment, Service, ConfigMap ve Secret dosyaları hazırlanmış, Dockerfile multi-stage olarak kurgulanmıştır.

5. Pipeline ve Dağıtım

CI hattında ruff lint kontrolü, pytest coverage kontrolü, Docker image build, smoke test ve Newman koleksiyonu çalıştırılmaktadır.

Kubernetes tarafında Deployment, Service, ConfigMap ve Secret dosyaları hazırlanmıştır. Docker Compose ile yerel ortamda PostgreSQL, LocalStack, Prometheus ve Grafana birlikte ayağa kaldırılabilmektedir.

Yerel raporlama için mimari PNG, final rapor PDF ve slayt PDF çıktıları otomatik üretilmiştir.

6. Performans ve Gözlemlenebilirlik

Prometheus uygulamanın `/metrics` endpoint'ini scrape etmektedir. Grafana dashboard'unda throughput, error rate ve p95 latency panelleri tanımlanmıştır.

Yük testi akışı `GET /menu`, `POST /orders` ve `PATCH /orders/{id}/status` uçlarını hedeflemektedir.

Yerel Locust ölçümünde p95 latency 16 ms, maksimum gecikme 30 ms ve hata oranı yüzde 0 olarak gözlemlenmiştir.

Toplam 66 isteklik ölçümde ortalama gecikme 7 ms ve throughput değeri 4.63 req/s olarak kaydedilmiştir.

7. Karşılaşılan Zorluklar ve Öğrenilenler

En önemli teknik öğrenimlerden biri Testcontainers ile kullanılan PostgreSQL sürücüsünün projedeki `psycopg` bağımlılığıyla hizalanması olmuştur.

Ayrıca ortama bazı testlerin kontrollü skip edilmesi, gözlemlenebilirlik dosyalarının erken hazırlanması ve Kubernetes secret ayrımı proje kalitesini artırmıştır.

Playwright ve performans testlerinde bazı ortamlarda port ve browser erişimi kısıtlı olabildiği için testler gerektiğinde güvenli şekilde skip olacak biçimde tasarlanmıştır.

8. Sonuç

Proje, artnamedeki mini servis, test, container, LocalStack, veritabanı, monitoring ve performans katmanlarının büyük bölümünü tamamlamış durumdadır.

Özellikle CI entegrasyonu, arşivleme davranışları, ölçümleme altyapısı ve teslimat dosyalarının otomatik üretilmesi projeyi sunuma hazır hale getirmiştir.

9. Kaynaklar

FastAPI, SQLAlchemy, Testcontainers, LocalStack, Prometheus, Grafana ve k6 dokümantasyonları proje boyunca başlıca referanslar olarak kullanılmıştır.